

Document Chunker Validation Sample

Overview

This sample document is designed to validate a context-aware document chunking pipeline. It contains several short sections, neighboring paragraphs, policy-like content, and repeated search terms. The goal is to make it easy to test parsing, chunk creation, metadata preservation, keyword search, and context expansion.

The document chunker should preserve the source file name, page locator, heading path, and neighboring chunk relationships. When a search result is returned, the caller should be able to understand where the text came from and how it relates to nearby content.

Project Background

The document chunker is a small local module for importing documents, splitting them into meaningful chunks, storing those chunks, and returning structured search results. It is intentionally simpler than a full retrieval system. It does not train embeddings, manage a vector database, or perform semantic ranking.

The first version focuses on a deterministic workflow. A local document is parsed into normalized blocks. The blocks are split into chunks without crossing obvious section boundaries. The chunks are written to a local store. A search command can then return matching chunks and optional context.

Chunking Requirements

A good chunk should be large enough to carry useful meaning but small enough to fit into a downstream retrieval or language model workflow. The chunker should prefer sentence boundaries instead of cutting a sentence in the middle. It should also preserve neighboring relationships so that a caller can recover the previous and next chunks.

Section boundaries matter. Text under one heading should not silently merge with text under

another heading. If the document has a heading called Retention Policy, chunks under that heading should keep Retention Policy in their heading path. This makes the output easier to cite and easier to debug.

Retention Policy

Policy records are retained for seven years. Search should find retention policy text in this section. The phrase retention policy appears more than once so that keyword search can be tested reliably.

Retention rules are important because downstream users often ask questions such as how long a document should be kept, who owns it, and where it should be stored. A useful chunking system should return the policy text together with enough context to explain why the answer is relevant.

Access Control

Only authorized users should access indexed documents. The chunk store is local in this demo, but the same design should make it clear where authentication, authorization, and audit logging would be added in a production system.

Access control metadata is not implemented in this sample document. However, the output format leaves room for metadata fields so that future versions can include owner, department, classification, or retention category.

Table Context Example

The following lines simulate table-like content in plain text.

Department: Legal
Owner: Records Team
Retention Period: Seven years
Review Frequency: Annual
Escalation Contact: Compliance Lead

A parser for spreadsheet files should preserve sheet names and column headers. A parser for PDF files may not always recover table structure perfectly, but it should still preserve page-level

location and extractable text.

Search Scenarios

Search for the phrase retention policy. The expected result should come from the Retention Policy section.

Search for the phrase access control. The expected result should come from the Access Control section.

Search for the phrase downstream users. The expected result should return a chunk that explains how retained policy text may be used by another retrieval or question-answering component.

Conclusion

This PDF is only a small demo file, but it exercises the most important path of the document chunker. It contains headings, paragraphs, repeated keywords, policy language, and context that can be restored during search. If ingest, search, export, and context expansion all work on this document, the main demo path is healthy.